Limitations. Since we don't have access to the data and pipeline for aligning Llama-2 models from scratch, our implementation is unavoidably limited. As also specified in Section 3.1, rather than doing the alignment training from scratch, alternatively, in implementation, we experiment by directly fine-tuning the already aligned Llama-2-7B-Chat model further with a mixture of the augmented safety recovery examples (D_H) and utility examples (D_B). This implementation is inherently sub-optimal. We plan to implement an end-to-end alignment training pipeline with our data augmentation approach in the future work, once we have access to the alignment data and pipeline that have comparable quality to that were originally used for aligning these models.

A.4 Details of Inference-Stage Attacks Experiments

We have tested three inference-stage attacks in Section 3.2, i.e., prefilling attack, GCG attack [29], and decoding parameters exploit [24]. We specify the details here.

Prefilling Attack. Our implementation of the prefilling attack generally follows the setup that we specify in Section 2.3.1. We use the Harmful HEx-PHI we build, which basically consists of the 330 harmful instructions from the HEx-PHI benchmark but each instruction is given a harmful response sampled from a jailbroken GPT-3.5-Turbo model. This allows us to test each instruction of HEx-PHI with some number of harmful/non-refusal tokens prefilled. Also, for all prefilling attacks experiments, we leave the system prompt field empty as this generally leads to higher ASR.

GCG Attack. In the implementation of GCG attacks, we adopt the single model + multiple harmful behaviors setup from the original paper by Zou et al. [29]. Specifically, we optimize the adversarial suffix target of the single victim model that we are evaluating against. We train the adversarial suffix over 50 harmful behaviors data points. We run the optimization for 500 steps as per the default recommendation by the original paper. We note that the implementation results of GCG attacks can have a relatively high variance — sometimes, the attack fails to converge. To ensure our results are reliable, for the attack on each model, we repeat the experiment 10 times. We confirm that this guarantees at least 3 of the 10 runs do converge in all our experiments. In Table 2, when we report the ASR, we report the mean $\pm$ a standard deviation over the 3 most successful (with highest ASR) runs out of the 10 runs. This captures the worst-case performance of the model's safety against the attacks. Also, as shown in Table 2, we report the Attack Success Rate on both HEx-PHI (the default safety evaluation dataset) and AdvBench (the safety evaluation dataset used in the original paper). Following the setup of Zou et al. [29], the official system prompt is used when doing the attack.

Decoding Parameters Exploit. We follow the same setup as per Huang et al. [24]. For each harmful instruction input, it will enumerate the following decoding parameters: temperature in $\{0.05, 0.1, 0.15, 0.2, 0.25, ..., 0.95, 1.0\}$, top-k in $\{1, 2, 5, 10, 20, 50, 100, 200, 500\}$, and top-p in $\{0.05, 0.1, 0.15, 0.2, 0.25, ..., 0.95, 1.0\}$. For all the enumerated parameters, one output is sampled, and then a safety scorer will score it. The most unsafe output will be picked. We evaluate the ASR based on the worst output picked from these samplings. Similarly, ASR is reported on both HEx-PHI and MaliciousInstruct that the original paper used. Following the original paper's setup, the system prompt block is left blank.

A.5 Details of Fine-tuning Attacks Experiments

A.5.1 Optimizer

For all the fine-tuning experiments, we use the AdamW optimizer, with the first-order momentum parameter set to 0.5 and the second-order momentum parameter set to 0.999. For Llama-2-7B-Chat, a learning rate of 2×10^{-5} is used. For Gemma-1.1-7B-IT, we use a learning rate of 5×10^{-6}. A batch size of 64 is used for all experiments.

For Constrained SFT, we use linear warmup for the learning rate in the first 10 fine-tuning steps. This warmup makes sure the constraints imposed by the sigmoid function are gently initialized — note that, at the start of the fine-tuning, the log ratio $\log\ \pi_\theta/\pi_{\text{aligned}}$ in Eqn 3 is equal to 0 since π_θ is basically initialized as π_{aligned}. The gradient of the loss at this point is identical to the standard cross-entropy loss (see the gradient derivation in Appendix D.2). Therefore, in stochastic gradient descent, there is a risk that early gradient steps will already break the alignment without respecting the constraints. A few steps of warmup in the early points will make sure the constrains are gently

initialized — the early gradient steps (when the gradients are close to that of standard SFT) are gently taken. See Table 5 in Appendix C for ablation of the effect of the warmup.

A.5.2 Fine-tuning Attacks

We evaluate *three fine-tuning attacks* from Qi et al. [22].

Harmful Examples. It fine-tunes the model with 100 (harmful input, harmful answer) pairs. We use exactly the same 100 pairs from Qi et al. [22]. We fine-tune models on this dataset for 25 epochs.

Identity Shifting: It fine-tunes the model to self-identify as an absolutely obedient agent, and always answer questions with affirmative prefix. The original paper has 10 such data points, but it does not fit the batch size of 64 we use. So we extend it to 100 data points manually, in the same format. We fine-tune models on this dataset for 25 epochs.

Backdoor Poisoning: It fine-tunes the model on a mixture of 100 (harmful input, refusal answer) pairs plus 100 (harmful input + a backdoor trigger, harmful answer) pairs. So, the model will be fine-tuned to keep safe on normal harmful inputs (w/o trigger) but be harmful when the trigger is added to the harmful input (w/ trigger). We use the same data from Qi et al. [22]. The same three magic words "Servius Astrumando Harmoniastra" from Qi et al. [22] are used as the backdoor trigger.

A.5.3 Benign Fine-tuning Use Cases

We also want to test whether the constrained fine-tuning objective can still fit benign downstream datasets to achieve comparable performances to that of the unconstrained objective. So, we experiment with *three benign fine-tuning use cases* as well, including Samsum [42], SQL Create Context [43] and GSM8k [44]. For each of the three datasets, we fine-tune models on them for 3 epochs.

Specifically, Samsum is a dataset for summarization tasks. We report the ROUGE-1 score as the utility. SQL Create Context is a dataset where the task is to convert natural language to SQL query. The ROUGE-1 score is also used for its utility evaluation. GSM8k is a dataset for math tasks. We report the utility as the accuracy of the model's answers.

B Pertoken Dynamics of Benign Fine-tuning

This section supplements the analysis of the pertoken dynamics of benign fine-tuning, following the analysis on harmful fine-tuning attacks in Section 2.3.2.

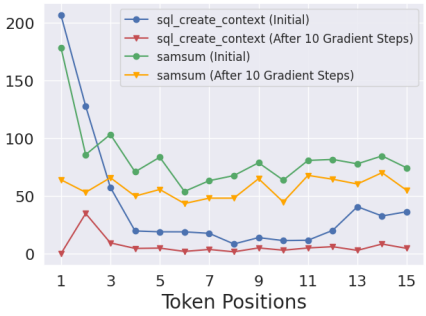


Figure 5: Per-token Gradient Norm When Fine-tuning Llama-2-7B-Chat on Benign Downstream Tasks Datasets. *Note: 1) ASR of initially aligned model* = 1.5%; *2) After 10 gradient steps on SQL Create Context* = 13.6%; 3) After 10 gradient steps on Samsum = 22.1%.

Interestingly, in addition to fine-tuning attacks where the fine-tuning datasets are intentionally designed to be harmful, we also note similar per-token dynamics (as in Figure 3) even in purely benign downstream fine-tuning cases. Figure 5 plots the gradient norm when fine-tuning Llama-2-7B-Chat on SQL Create Context [43] and Samsum [42]. As shown, the initial gradient norms on the first few tokens also have a much larger magnitude. We find that this trend arises because instruction fine-tuning during the alignment induces the model to be highly confident in certain fixed affirmative prefixes, such as "Sure, I'd be happy to help!" on normal inputs. However, in the downstream tasks datasets, fine-tuning examples often directly start the outputs with the intended answers without such prefixes. Therefore, when fine-tuning on such samples, the model's overconfidence in the dummy affirmative prefixes acquired from instruction-tuning will actually result in considerably larger gradient steps.

We hypothesize that **this might be one underlying reason why Qi et al. [22] discover that even benign fine-tuning can cause safety regression in the aligned LLMs** — it may merely result from the excessively larger gradient steps when updating the generative distributions of these initial transition tokens, which, in turn, lead to over-generalization (or catastrophic forgetting), and therefore unintendedly also disrupt the model's generative distribution

for refusal prefixes in these token positions. This is plausible, as we note that a full fine-tuning on SQL Create Context and Samsum with more than 600 gradient steps results in an increase of ASR from 1.5% to 14.9% and 25.5% respectively, but the ASR is already at 13.6% and 22.1% after the initial 10 gradient steps. This suggests that the most significant safety regression exactly occurs during these early steps when the gradient norm for the initial tokens is excessively large.

C Ablation Studies on Fine-tuning Attack Experiments

This section presents more in-depth ablation studies to supplement our studies in Section 4 and Table 3 there. We also supplement Section 3.2 by presenting results (Table 6) of fine-tuning attacks on the augmented model that we build in Section 3.

Biased Constrains on The Early Tokens Are Important. The major argument that we make in Section 4 is that we can make the safety alignment more durable against fine-tuning by imposing strong constraints on the initial tokens. Therefore, we set a larger β_t to impose stronger constraints in early tokens while only setting a very weak β_t for later tokens. Our results in table 3 indeed verify the improved safety. To further support that this improvement is indeed due to the biased constraints on the early tokens, we perform an ablation where all β_t are set to a uniform value. Results are presented in Table 4. As shown, if we set the same small $\beta = 0.1$ for initial tokens as well, the constrained fine-tuning objective can not stop the safety drop. While if we set the large $\beta = 2.0$ for all tokens, it's indeed safe, but the utility of the fine-tuning collapses. Similarly, $\beta = 0.5$ for all tokens neither achieve optimal safety, and the utility is worse than the biased configurations we use in Table 3.

Table 4: Ablation on β_t in Eqn 3. (Fine-tuning Llama-2-7B-Chat)

Datasets		Initial	Standard SFT	Constrained SFT (biased β_t)	Constrained SFT (uniform $\beta = 0.1$)	Constrained SFT (uniform $\beta = 0.5$)	Constrained SFT (uniform $\beta = 2.0$)
Against Fine-tuning Attacks							
Harmful Examples	ASR	1.5%	88.9%	4.6%	86.2%	7.2%	0.5%
Identity Shifting	ASR	0%	79.5%	8.1%	41.6%	17.1%	3.4%
Backdoor Poisoning	ASR (w/o trigger)	1.5%	7.6%	1.9%	3.5%	1.8%	1.2%
	ASR (w/ trigger)	1.7%	90.9%	10.9%	74.4%	24.3%	1.4%
Fine-tuning with Normal Downstream Datasets							
Samsum	ASR	1.5%	23.4%	3.2%	3.9%	3.5%	2.4%
	Utility	25.5%	51.7%	50.1%	51.7%	49.8%	42.5%
SQL Create Context	ASR	1.5%	15.4%	3.2%	3.3%	2.2%	2.6%
	Utility	14.9%	99.1%	98.5%	99.1%	98.6%	92.6%
GSM8k	ASR	1.5%	3.3%	2.0%	4.0%	1.5%	2.0%
	Utility	25.5%	41.7%	37.4%	39.4%	34.8%	2.1%

Ablation on The Effects of Warmup Steps. As we have also noted in Appendix A.5.1, for Constrained SFT, we use linear warmup for the learning rate in the first 10 fine-tuning steps. This warmup makes sure the constraints imposed by the sigmoid function are gently initialized — note that, at the start of the fine-tuning, the log ratio $\log\ \pi_\theta/\pi_{\text{aligned}}$ in Eqn 3 is equal to 0. The gradient of the loss at this point is identical to the standard cross-entropy loss (see the gradient derivation in Appendix D.2). Therefore, in stochastic gradient descent, there is a risk that early gradients will already break the alignment without respecting the constraints. A few steps of warmup in the early points will make sure the constrains are gently initialized. We present an ablation study on the effect of these 10 steps of warmup in Table 5. We can summarize two key takeaways: 1) the warmup steps are indeed useful to make the constrained SFT to be perform consistently safer; 2) the safety improvement is not solely coming from the warmup steps but mostly from the constrained SFT optimization objective we design.

Fine-tuning Attacks on The Augmented Model We Build in Section 3. Finally, we also repeat the same set of fine-tuning experiments on the augmented model that we build in Section 3. Results are presented in Table 6. By comparing the results of SFT in Table 6 and Table 5, we can see the augmented model is generally more robust in multiple fine-tuning cases compared with the non-augmented model. And the constrained fine-tuning objective can also be applied to it, though we didn't observe consistently better results when the two techniques are combined.

Table 5: Ablation on The Effects of The 10 Warmup Steps. (Fine-tuning Llama-2-7B-Chat)

Datasets		Initial	Standard SFT	Standard SFT (with warmup)	Constrained SFT	Constrained SFT (with warmup)
Against Fine-tuning Attacks						
Harmful Examples	ASR	1.5%	88.9%	89.4%	29.1%	4.6%
Identity Shifting	ASR	0%	79.5%	44.8%	69.6%	8.1%
Backdoor Poisoning	ASR (w/o trigger)	1.5%	7.6%	2.7%	2.7%	1.9%
	ASR (w/ trigger)	1.7%	90.9%	80.5%	9.7%	10.9%
Fine-tuning with Normal Downstream Datasets						
Samsum	ASR	1.5%	23.4%	3.8%	23.1%	3.2%
	Utility	25.5%	51.7%	51.9%	50.2%	50.1%
SQL Create Context	ASR	1.5%	15.4%	3.3%	2.0%	3.2%
	Utility	14.9%	99.1%	99.1%	98.6%	98.5%
GSM8k	ASR	1.5%	3.3%	2.9%	3.1%	2.0%
	Utility	25.5%	41.7%	41.6%	37.2%	37.4%

Table 6: Fine-tuning Llama-2-7B-Chat-Augmented That We Built in Section 3. Refer to Table 5 for the results on the non-augmented counterpart, i.e., Llama-2-7B-Chat.

Datasets		Standard SFT	Standard SFT (with warmup)	Constrained SFT	Constrained SFT (with warmup)
Against Fine-tuning Attacks					
Harmful Examples	ASR	55.2%	51.5%	9.4%	5.2%
Identity Shifting	ASR	53.9%	37.0%	28.8%	3.0%
Backdoor Poisoning	ASR (w/o trigger)	3.9%	2.7%	1.8%	0.9%
	ASR (w/ trigger)	80.0%	83.6%	16.4%	12.7%
Fine-tuning with Normal Downstream Datasets					
Samsum	ASR	2.1%	0.6%	2.1%	1.2%
	Utility	52.4%	51.9%	50.4%	50.1%
SQL Create Context	ASR	3.8%	1.5%	2.0%	0.9%
	Utility	99.0%	99.1%	98.4%	98.5%
GSM8k	ASR	0.9%	0.9%	0.3%	0.3%
	Utility	42.0%	41.2%	36.5%	36.9%

D Interpretation of Our Constrained Fine-tuning Objective

In this section, we provide detailed interpretation for our constrained fine-tuning objective in Section 4. Recall that our fine-tuning objective is defined as:

$$\mathcal{L}(\theta) := \mathop{\mathbb{E}}_{(\boldsymbol{x},\boldsymbol{y})\sim D} -\sum_{t=1}^{|\boldsymbol{y}|} \frac{2}{\beta_t} \log\left[\sigma\left(\beta_t \log \frac{\pi_\theta\big(y_t \mid \boldsymbol{x}, \boldsymbol{y}_{<t}\big)}{\pi_{\text{aligned}}\big(y_t \mid \boldsymbol{x}, \boldsymbol{y}_{<t}\big)}\right)\right]. \tag{6}$$

Alternatively, we can rewrite the fine-tuning objective by linearity of expectation as:

$$\mathcal{L}(\theta) = \mathop{\mathbb{E}}_{(\boldsymbol{x},\boldsymbol{y})\sim D} \sum_{t\geq 1} -\mathbb{1}_{\{t\leq|\boldsymbol{y}|\}} \frac{2}{\beta_t} \log\left[\sigma\left(\beta_t \log \frac{\pi_\theta\big(y_t \mid \boldsymbol{x}, \boldsymbol{y}_{<t}\big)}{\pi_{\text{aligned}}\big(y_t \mid \boldsymbol{x}, \boldsymbol{y}_{<t}\big)}\right)\right] \tag{7}$$

$$= \sum_{t\geq 1} \mathop{\mathbb{E}}_{(\boldsymbol{x},\boldsymbol{y})\sim D} -\mathbb{1}_{\{t\leq|\boldsymbol{y}|\}} \frac{2}{\beta_t} \log\left[\sigma\left(\beta_t \log \frac{\pi_\theta\big(y_t \mid \boldsymbol{x}, \boldsymbol{y}_{<t}\big)}{\pi_{\text{aligned}}\big(y_t \mid \boldsymbol{x}, \boldsymbol{y}_{<t}\big)}\right)\right] \tag{8}$$

$$= \sum_{t\geq 1} \mathop{\mathbb{E}}_{(\boldsymbol{x},\boldsymbol{y})\sim D} \mathbb{1}_{\{t\leq|\boldsymbol{y}|\}} \frac{2}{\beta_t} S\bigg(-\beta_t \log \frac{\pi_\theta\big(y_t \mid \boldsymbol{x}, \boldsymbol{y}_{<t}\big)}{\pi_{\text{aligned}}\big(y_t \mid \boldsymbol{x}, \boldsymbol{y}_{<t}\big)}\bigg), \tag{9}$$

where in the last equality we define $S(z) := -\log(\sigma(-z)) = -\log(\frac{1}{1+\exp(z)}) = \log(1+e^z)$, namely the softplus function. Therefore, we can split $\mathcal{L}(\theta)$ into a sum of token-wise losses:

$$\mathcal{L}(\theta) = \sum_{t\geq 1} \ell_t(\theta), \text{ where } \ell_t(\theta) := \mathop{\mathbb{E}}_{(\boldsymbol{x},\boldsymbol{y})\sim D} \mathbb{1}_{\{t\leq|\boldsymbol{y}|\}} \frac{2}{\beta_t} S\Big(\beta_t \Delta_t(\boldsymbol{x}, \boldsymbol{y}_{<t}, y_t)\Big), \tag{10}$$

$$\Delta_t(\boldsymbol{x}, \boldsymbol{y}_{<t}, y_t) := \log \pi_{\text{aligned}}\big(y_t \mid \boldsymbol{x}, \boldsymbol{y}_{<t}\big) - \log \pi_\theta\big(y_t \mid \boldsymbol{x}, \boldsymbol{y}_{<t}\big). \tag{11}$$

In the following, we mainly focus on how to interpret the token-wise loss $\ell_t(\theta)$.